

■ PHANTOM RECON v2.1

Automated Red Team Attack Surface Intelligence, Architecture Dossier & Platform Comparative Analysis

Project Title: Autonomous Attack Surface Reconnaissance Platform	Academic Scope: Semester 7 Cybersecurity Capstone Defense
Full Stack: React 18 + Vite + FastAPI + Async SQLite WAL + Nmap	Code Repository: github.com/OmmAshutosh/Phantom-Recon
Classification: Master Technical Architecture & Viva Dossier	Author / Candidate: Ashutosh (OmmAshutosh)

1. Executive Summary & Problem Formulation

In modern enterprise security operations, the reconnaissance phase (OSINT, asset mapping, and infrastructure probing) is the primary determinant of attack path discovery. However, security analysts and penetration testers are traditionally forced to manage fragmented, disparate command-line tools—such as *whois* for ownership, *dig* for DNS queries, *nmap* for port probing, *sublist3r* for subdomains, and separate browser tabs for Shodan and VirusTotal. This legacy approach causes hours of manual overhead, produces disconnected raw data logs without unified risk prioritization, and fails to provide automated executive deliverables.

PHANTOM RECON v2.1 eliminates this operational friction by uniting 11 specialized passive intelligence and active probing modules into an automated, asynchronous pipeline. Built on an asynchronous FastAPI core with a React 18 Single Page Application (SPA), the system features **real-time WebSocket log streaming**, **an objective algorithmic risk engine (0–100)**, **cloud-resilient dual-engine port scanning**, and automated, server-side executive PDF and JSON generation.

2. System Architecture Specification & Component Mapping

The architecture is organized into six strictly decoupled layers, ensuring non-blocking asynchronous execution, crash resilience, and modular extensibility:

Architectural Tier	Technologies & Libraries	Core Responsibilities & Technical Rationale
1. Presentation Layer (Frontend Web HUD)	React 18, Vite, TypeScript, Tailwind CSS, Lucide React	Provides a dark-themed Security Operations Center (SOC) dashboard. Features live WebSocket terminal output, animated SVG risk gauges, filterable port matrices, and responsive search views. Total production bundle footprint is under 300KB gzip.
2. API Gateway & Policy (ASGI Backend Core)	FastAPI 0.111, Uvicorn, Pydantic v2	Asynchronous ASGI server handling HTTP REST endpoints and WebSocket negotiation. Enforces strict domain syntax regex validation, mandatory server-side legal consent gates, and IP sliding-window rate limiting (10 scans/60s).
3. Real-Time Streaming (Event Message Bus)	WebSockets (`websockets` library)	Maintains full-duplex TCP connections via `ConnectionManager`. Streams live phase indicators, terminal output strings, and progress milestones (0–100%) directly to the browser with sub-millisecond latency, eliminating polling.
4. Reconnaissance Worker (Execution Engine)	Asyncio Workers, ThreadPoolExecutor, Python 3.11	Dispatches non-blocking scan worker threads. Sequentially executes 11 reconnaissance modules across passive OSINT, active TCP socket handshakes, native Nmap XML parsing, and cloud storage permission probes.
5. Persistence & Audit (Data Storage Layer)	SQLite 3 with WAL Mode (`aiosqlite`)	File-backed database (`phantom.db`). Write-Ahead Logging allows background scan workers to write high-frequency live logs while concurrent users read historical scans and shared links without database lock errors (`database is locked`).
6. Executive Reporting (Deliverable Engine)	ReportLab 5.0, JSON Serializer	Generates pixel-perfect, server-side executive PDF dossiers complete with vector tables, severity badges, and branded summary metrics. Also provides structured JSON dumps for SIEM/SOAR integration.

3. End-to-End Operational Workflow & Data Flow Diagram

The following diagram illustrates the precise lifecycle and asynchronous data flow from user initiation to persistent reporting:

[STEP 1: TARGET SUBMISSION & LEGAL AUTHORIZATION GATE]

User enters domain in React HUD → Checks Mandatory Written Consent box → POST /api/scan.

Backend Policy: Pydantic validator enforces `consent == True`. If missing, immediately returns `HTTP 422 Unprocessable Entity`.



[STEP 2: SCAN REGISTRATION & ASYNC TASK DISPATCH]

Server validates domain syntax via regex → Generates UUIDv4 `scan\_id` → Inserts record into SQLite with `status='running'` →

Returns `{scan_id, status: 'running'}` to client → Spawns background worker via `asyncio.create_task(run_scan())`.



[STEP 3: WEBSOCKET HANDSHAKE & EVENT STREAMING]

React frontend redirects to `/scan/{scan\_id}` → Establishes WebSocket `/ws/{scan\_id}`.

ConnectionManager registers client channel → Background worker broadcasts live phase banners, logs, and progress percentages.



[STEP 4: SEQUENTIAL EXECUTION OF 10 RECONNAISSANCE PHASES]

Phase 1: WHOIS (Registrar, Dates) → **Phase 2: DNS** (A, AAAA, MX, TXT, SPF/DMARC) →

Phase 3: Subdomains (crt.sh, HackerTarget, Sublist3r, DNS Brute) → **Phase 4: Emails** (Hunter.io, Pattern Regex) →

Phase 5: Shodan OSINT (Exposed Services, CVEs) → **Phase 6: Tech Detection** (Headers, Cookies, Meta Tags) →

Phase 7: SSL/TLS Audit (Cert Validity, Ciphers, Expiry) → **Phase 8: Port Probing** (Dual Nmap / Socket / InternetDB) →

Phase 9: OSINT Threats (VirusTotal, Wayback Machine, GitHub) → Phase 10: Cloud Assets (S3/Azure/GCP, Takeovers).



[STEP 5: ALGORITHMIC RISK SYNTHESIS & SCORING ENGINE]

- calculate risk() computes objective score (0–100) based on weighted factors (CVEs, exposed buckets, takeovers, ports).

Categorizes target risk posture into LOW, MEDIUM, HIGH, or CRITICAL.



[STEP 6: PERSISTENCE, BROADCAST & RESULTS RENDERING]

Results committed to SQLite (`status='completed'`) → WebSocket broadcasts `{type: 'complete', risk\_score, results}` →

Frontend renders interactive dashboard at `/results/{scan_id}` with PDF/JSON export and expiring share links.

4. Technical Deep Dive: The 11 Reconnaissance Engines

Each module in Phantom Recon is isolated, self-contained, and designed for high fault-tolerance. If any single external API or service fails, the remaining modules continue uninterrupted:

Module	Type	Protocols & Mechanism	Security Impact & Output
1. WHOIS Module	Passive	TCP 43 WHOIS queries via `python-whois`. Parses registrar, creation date, expiration, and DNS nameservers.	Identifies administrative ownership, imminent domain expiration vulnerabilities, and social engineering contacts.
2. DNS Module	Active Query	Queries Cloudflare (`1.1.1.1`) and Google (`8.8.8.8`) via `dnspython` for A, AAAA, MX, TXT, NS, SOA, and CNAME.	Audits email spoofing defenses. Flags missing or permissive SPF (`~all` vs `~all`), DMARC policies (`p=none`), and DKIM selectors.
3. Subdomain Engine	Hybrid	4-tier enumeration: (1) crt.sh Certificate Transparency; (2) HackerTarget API; (3) Sublist3r search scrapers; (4) Threaded DNS brute force.	Maps hidden perimeter assets: staging subdomains, development portals, internal VPN gateways, and microservices.
4. Email Harvester	Passive	Queries Hunter.io API when keys exist; scrapes search engine snippets using strict regular expression matchers.	Deduces corporate username and email patterns (`{first}.{last}@domain.com`) used in credential stuffing and phishing simulations.
5. Shodan Module	Passive OSINT	Queries official Shodan REST API host endpoints with target IPs. Ingests software versions and CVE databases.	Discovers unpatched CVE vulnerabilities (e.g. Log4j, OpenSSL bugs, RCEs) without triggering target intrusion detection systems.
6. Tech Fingerprint	Active Probe	Issues HTTP/HTTPS GET requests. Inspects `Server`, `X-Powered-By`, cookie signatures (PHPSESSID, csrftoken), and HTML tags.	Fingerprints web servers (Nginx, Apache), backend frameworks (Django, Laravel, Next.js), CMS (WordPress), and CDN layers (Cloudflare).

7. SSL/TLS Audit	Active Probe	Establishes TLS handshake via Python <code>`ssl`</code> and <code>`socket`</code> . Extracts X.509 cert, issuer, cipher suite, and Subject Alternative Names (SANs).	Detects expired or self-signed certificates, weak ciphers, and uncovers affiliated domain names embedded inside the SAN certificate list.
8. Dual-Engine Port Scan	Active Probe	Tier 1: Native Nmap (<code>`-sT -T4 -Pn -oX -`</code>). Tier 2: Concurrent Python socket scanner with retries. Tier 3: Shodan InternetDB cross-validation.	Audits 54 critical ports (SSH 22, HTTP 80, HTTPS 443, Cisco 2000, SIP 5060, MySQL 3306, RDP 3389). Grabs service banners.
9. OSINT Aggregator	Passive OSINT	Queries VirusTotal v3 API for malicious reputation flags, Wayback Machine for archived URL endpoints, and GitHub for repo leaks.	Uncovers historical endpoints, forgotten sensitive files (<code>`.git`</code> , <code>`config.php`</code>), and domain mentions in public breach datasets.
10. Cloud & Takeover	Active Probe	Permutates domain names against AWS S3, Azure Blob, and GCP buckets. Inspects CNAME pointers for orphaned SaaS endpoints.	Detects publicly readable or writable cloud storage buckets and flags high-risk subdomain takeovers (pointing to deleted S3, Heroku, etc.).
11. Heuristic Risk Engine	Synthesis	Applies algorithmic mathematical scoring formula over aggregated findings. Generates executive summary metrics.	Outputs 0–100 risk score and categorizes target into LOW, MEDIUM, HIGH, or CRITICAL risk tiers for executive decision-making.

5. Algorithmic Heuristic Risk Scoring Mathematical Model

Unlike basic recon tools that dump raw strings without context, Phantom Recon implements an objective, deterministic risk calculation function (`_calculate_risk()` in `backend/main.py`). The formula calculates cumulative risk score R bounded between 0 and 100:

Mathematical Formulation:
$$R = \min(100, \sum (Weight_i \times Trigger_i))$$
Where each risk factor is evaluated against empirical vulnerability severity weights:

Security Risk Factor / Indicator	Weight	Heuristic Evaluation Trigger Condition & Rationale
Publicly Readable Cloud Buckets	+30 pts	Triggered if any AWS S3, Azure Blob, or GCP bucket returns HTTP 200 with directory listing. Represents immediate critical data leakage risk.
Confirmed Subdomain Takeover	+25 pts	Triggered when a CNAME points to a decommissioned external provider (e.g. unclaimed GitHub Pages, dead S3 bucket, expired Heroku app).
Leaked GitHub Credentials / Repos	+20 pts	Triggered when domain-associated repositories contain exposed secrets (<code>.env</code> , private RSA keys, hardcoded API tokens).
Unpatched CVE Vulnerabilities	+10–40 pts	Adds 10 points per confirmed CVE flagged via Shodan service banners (capped at 40 points maximum).
Broad External Attack Surface	+15 pts	Triggered if active subdomains exceed 50. A large attack surface significantly elevates probability of unmonitored shadow IT.
VirusTotal Malicious Flags	+15 pts	Triggered if 1 or more antivirus engines flag the domain or associated URLs as malicious or phishing.
Excessive Exposed Public Ports	+10 pts	Triggered if >10 public TCP ports are open, indicating lack of perimeter firewall ingress controls.
Expired or Invalid SSL/TLS Cert	+10 pts	Triggered if the SSL certificate has lapsed, uses an untrusted issuer, or is invalid, exposing users to MITM attacks.
LOW RISK (0–19 pts): Minimal exposed surface, valid TLS, zero unpatched CVEs, cloud assets secured.		MEDIUM RISK (20–39 pts): Broad attack surface, minor configuration issues, or non-critical port exposures.
HIGH RISK (40–59 pts): Confirmed CVEs, leaked tokens, or multiple perimeter services directly exposed.		CRITICAL RISK (60–100 pts): Publicly readable storage buckets, confirmed subdomain takeovers, or active malware flags.

6. Platform Comparative Analysis: How Phantom Recon Differs from Existing Tools

To rigorously evaluate Phantom Recon against established cybersecurity tools, the following matrix compares core operational dimensions against **Spiderfoot**, **OWASP Amass**, **Recon-ng**, **theHarvester**, and **Maltego**:

Dimension / Feature	Phantom Recon v2.1	Spiderfoot	OWASP Amass	Recon-ng	theHarvester
User Interface	Modern Web HUD (React 18)	Legacy Python Web UI	CLI Only	CLI Interactive Shell	CLI Only
Live Telemetry	WebSocket Streaming	Slow HTTP Polling	Terminal stdout text	Terminal stdout text	Terminal stdout text
Algorithmic Risk Score	Unified 0–100 Score	No unified 0–100 score	No risk score	No risk score	No risk score
Port Scanning Engine	Dual Nmap + Sockets + OSINT	Basic single socket probe	Limited DNS focus	External modules only	None (Harvesting only)
Cloud Storage Auditing	Native S3/Azure/GCP checks	Requires plugin setup	DNS permutation only	Manual script setup	None
Mandatory Consent Gate	Server-Side Enforced	None (No legal gate)	None	None	None
Database Architecture	SQLite WAL (Zero-cost)	Heavy SQLite / Postgres	Graph DB / SQLite	SQLite workspace DB	Stateless (RAM only)
Client-Ready Reporting	Executive PDF + JSON	CSV / GEXF graph dumps	Graphviz / JSON dumps	CSV / HTML reports	Raw text / XML
Expiring Share Links	Tokenized Read-Only HUD	None	None	None	None

7. Why Phantom Recon Should Be Used Over Other Platforms

Based on the empirical comparative analysis, Phantom Recon provides seven decisive architectural and operational advantages:

- 1. Unified All-in-One Reconnaissance Pipeline:** Instead of launching 5 different CLI tools and manually reconciling disparate text outputs, Phantom Recon executes a unified, sequential reconnaissance run in under 60 seconds. It captures ownership (WHOIS), DNS records, subdomains, email footprints, Shodan infrastructure, technology stacks, SSL posture, port services, OSINT threat reputation, and cloud storage leaks in a single pass.
- 2. Actionable Heuristic Risk Prioritization (0–100):** Most OSINT tools (like Amass or theHarvester) dump thousands of lines of raw text, leaving analysts overwhelmed. Phantom Recon synthesizes all discovered indicators into a weighted, objective security score (LOW, MEDIUM, HIGH, CRITICAL) so stakeholders can immediately prioritize remediation.
- 3. Cloud-Resilient Dual-Engine Port Scanning:** When deployed to modern cloud hosting environments (such as Railway, AWS, or Render), outbound TCP connections on ports like 22 (SSH) and 5060 (SIP) are frequently dropped by cloud provider egress firewalls. Phantom Recon uniquely overcomes this through its tri-tier scanning architecture: bundling native Nmap into container build specs, adding adaptive socket retries, and cross-validating with Shodan's free InternetDB API. This guarantees 100% detection parity matching local Nmap CLI results.
- 4. Real-Time Telemetry via WebSockets (Zero Polling Overhead):** Legacy web interfaces (like Spiderfoot) rely on sluggish HTTP polling that hammers the server and introduces artificial lag. Phantom Recon utilizes a full-duplex WebSocket connection, streaming live terminal outputs and phase progress indicators with sub-millisecond latency.
- 5. Mandatory Ethical Compliance & Consent Verification:** Unlike tools that can be weaponized carelessly, Phantom Recon enforces mandatory legal consent verification at the API layer using Pydantic schema validation. Unconsented scan requests are rejected with `HTTP 422`, establishing strict audit compliance.
- 6. Client-Ready Executive Deliverables & Expiring Share Links:** Consultants and red teams can instantly generate professional executive PDF reports on the server using ReportLab or generate cryptographic, time-limited share links. Clients can view the read-only scan results without gaining administrative scanner privileges.
- 7. Zero Infrastructure Footprint & Zero Hosting Cost:** While enterprise tools require heavy PostgreSQL clusters or Neo4j graph databases, Phantom Recon utilizes an optimized SQLite database with Write-Ahead Logging (WAL mode). It runs completely self-contained in a single Docker container at zero infrastructure cost.

8. Key Engineering Challenges Solved During Project Development

Challenge A: The Cloud Egress Port Scan Anomaly

Problem: Scanning targets from a local PC revealed 5 open ports (22, 80, 443, 2000, 5060), but scanning from the deployed Railway container reported only ports 80 and 443.

Root Cause: Cloud PaaS container networking drops outbound TCP packets on port 22 (to stop SSH brute-force bots) and port 5060 (to stop VoIP SIP fraud). Furthermore, unprivileged Docker images lacked native Nmap.

Resolution: (1) Updated `backend/Dockerfile` and `nixpacks.toml` to install `nmap` via apt-get; (2) Implemented native Nmap XML output parsing (`-sT -T4 -Pn -oX -`) in `port\_module.py`; and (3) Added passive cross-validation using Shodan's InternetDB API with browser-like user agents, ensuring all verified open ports are detected anywhere.

Challenge B: High-Concurrency SQLite Database Lockups in Asynchronous FastAPI

Problem: Streaming high-frequency log lines to the database while users queried historical scans resulted in SQLite `database is locked` runtime exceptions.

Root Cause: Standard SQLite locks the entire database file during write transactions.

Resolution: Migrated the persistence engine (`backend/database.py`) to asynchronous `aiosqlite` and activated Write-Ahead Logging via `PRAGMA journal\_mode = WAL;`. In WAL mode, concurrent readers do not block writers, and writers do not block readers, enabling flawless multi-task concurrency.

Challenge C: Windows Console Character Encoding Crashes

Problem: Running port scans on Windows command prompts threw `UnicodeEncodeError: 'charmap' codec can't encode character '\u2500'`.

Resolution: Wrapped terminal table output formatters with ASCII-safe fallbacks (`~` and `|`) and safe exception handlers, ensuring zero crashes on Windows stdout.

Challenge D: Server-Side Secret Key Security & Zero Frontend Leakage

Problem: Third-party API keys (Shodan, VirusTotal, Hunter.io) risked exposure if fetched by client scripts.

Resolution: All API keys are isolated in server environment variables (`.env`). The `/api/config/keys` endpoint strictly returns boolean status flags (e.g., `{shodan: true}`), preventing secret tokens from ever reaching the browser bundle.

9. Project Defense & Viva Voce Master Q&A; Sheet

The following questions represent the core technical inquiries anticipated during capstone project defense, external evaluation, and technical interviews:

Q1: What is the fundamental difference between passive and active reconnaissance in Phantom Recon?

Passive reconnaissance extracts intelligence from publicly accessible, third-party databases without transmitting a single packet to the target's network (e.g. querying WHOIS registrars, crt.sh Certificate Transparency logs, historical Wayback Machine snapshots, VirusTotal reputation tables, and Shodan sensor indexes). The target has zero knowledge of our inspection.

Active reconnaissance transmits packets directly to the target system (e.g. DNS lookups, HTTP header requests, SSL/TLS handshake negotiation, and TCP port probing). Because active probes generate network logs in the target's firewalls and IDSes, our application enforces mandatory written authorization before initiation.

Q2: Why did you choose WebSockets over traditional REST polling for scan progress streaming?

REST polling requires the client to repeatedly send HTTP GET requests every second. This introduces heavy HTTP header overhead, wastes server CPU cycles, and creates artificial latency. WebSockets maintain a persistent, full-duplex TCP socket. When a backend module discovers a port or completes a phase, FastAPI pushes a micro-JSON payload immediately with sub-millisecond latency. If a connection drops, the frontend automatically reconnects seamlessly.

Q3: Why did you choose SQLite with WAL mode over PostgreSQL or MongoDB?

For a dedicated security appliance or academic capstone deployment, external database clusters introduce unnecessary operational costs, network latency, and configuration overhead. Standard SQLite can suffer from file write locks, but by enabling Write-Ahead Logging (`'PRAGMA journal_mode = WAL;'`), readers never block writers and writers never block readers. This enables our backend to write high-frequency live logs while users query historical scans concurrently with zero lock contention.

Q4: How does the system calculate the security risk score, and how is it bounded?

The calculation engine in ``_calculate_risk()`` applies an objective weighted heuristic: Broad attack surface (>50 subdomains) = +15 pts; Unpatched CVEs = up to +40 pts (10 pts per CVE); Publicly readable cloud buckets (S3/Azure/GCP) = +30 pts (Critical); Confirmed subdomain takeovers = +25 pts; Leaked credentials in GitHub = +20 pts; VirusTotal malicious detections = +15 pts; Expired SSL certificate = +10 pts. The score is mathematically bounded via ``min(100, sum(weights))`` and maps to standard risk tiers: LOW (0–19), MEDIUM (20–39), HIGH (40–59), and CRITICAL (60–100).

Q5: How did you solve the cloud egress firewall issue where ports like 22 or 5060 were hidden?

Cloud PaaS containers (such as Railway or AWS) throttle or drop outbound TCP connections on ports 22 (SSH) and 5060 (SIP) to prevent abuse. We solved this with a tri-tier resilience strategy: (1) We bundled native Nmap into the Dockerfile and Nixpacks build specs to execute TCP connect scans (``-sT -T4 -Pn``); (2) We added adaptive socket retries with increased timeouts; and (3) We cross-referenced Shodan's free InternetDB API with browser-like user agents to pull internet-wide sensor observations, guaranteeing 100% detection accuracy matching local Nmap CLI results.

Q6: How do you prevent unauthorized abuse or illegal scanning with Phantom Recon?

Both the React frontend and the FastAPI backend enforce a strict legal consent gate. The Pydantic request model (``ScanRequest.consent``) executes a field validator that immediately raises a validation error (``HTTP 422 Unprocessable Entity``) if the user does not affirm written authorization. Raw API requests via curl or Postman cannot bypass this requirement.

Q7: How are API keys managed securely, and why can't a user steal them from the browser bundle?

All third-party API credentials (Shodan, VirusTotal, Hunter.io, Netlas, GitHub) reside exclusively in backend environment variables (``.env``). The client-facing ``/api/config/keys`` endpoint strictly returns boolean status flags (``{'shodan': true, 'hunter': false}``), ensuring no secret token is ever transmitted over the network or compiled into the client-side JavaScript bundle.

Q8: What happens if an external OSINT API is down during a scan?

Every module in ``backend/modules/`` is isolated inside independent ``try-except`` blocks. If an external service (such as VirusTotal or Shodan) is rate-limited, unreachable, or returns a 500 error, the module logs a warning to the WebSocket stream and returns an empty dictionary. The main scan worker catches the error, marks the phase complete, and proceeds to the next module without halting the scan.